

Singly

```

#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node* next;
};

Node* createNode(int value)
{
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void insertBeginning(Node*& head, int value)
{
    Node* newNode = createNode(value);
    newNode->next = head;
    head = newNode;
}

void insertEnd(Node*& head, int value)
{
    Node* newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
        return;
    }

    Node* temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}

```

```

void insertMiddle(Node*& head, int value, int position)
{
    if (position <= 1 || head == NULL)
    {
        insertBeginning(head, value);
        return;
    }

    Node* temp = head;

    for (int i = 1; i < position - 1 && temp->next != NULL; i++)
    {
        temp = temp->next;
    }

    Node* newNode = createNode(value);
    newNode->next = temp->next;
    temp->next = newNode;
}

```

```

void deleteBeginning(Node*& head)
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    Node* temp = head;
    head = head->next;
    delete temp;
}

```

```

void deleteEnd(Node*& head)
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    if (head->next == NULL)
    {
        delete head;
    }
}

```

```

        head = NULL;
        return;
    }

    Node* temp = head;

    while (temp->next->next != NULL)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;
}

void deleteMiddle(Node*& head, int position)
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    if (position == 1)
    {
        deleteBeginning(head);
        return;
    }

    Node* temp = head;

    for (int i = 1; i < position - 1 && temp->next != NULL; i++)
    {
        temp = temp->next;
    }

    if (temp->next == NULL)
    {
        cout << "Invalid Position\n";
        return;
    }

    Node* delNode = temp->next;
    temp->next = delNode->next;

```

```

        delete delNode;
    }

void display(Node* head)
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    while (head != NULL)
    {
        cout << head->data << " -> ";
        head = head->next;
    }

    cout << "NULL\n";
}

int main()
{
    Node* head = NULL;

    insertEnd(head, 10);
    insertEnd(head, 20);
    insertEnd(head, 30);

    cout << "Initial List:\n";
    display(head);

    insertBeginning(head, 5);
    cout << "\nAfter Inserting 5 at Beginning:\n";
    display(head);

    insertMiddle(head, 15, 3);
    cout << "\nAfter Inserting 15 at Position 3:\n";
    display(head);

    insertEnd(head, 40);
    cout << "\nAfter Inserting 40 at End:\n";
    display(head);

    deleteBeginning(head);

```

```
cout << "\nAfter Deleting from Beginning:\n";  
display(head);
```

```
deleteMiddle(head, 3);  
cout << "\nAfter Deleting Position 3:\n";  
display(head);
```

```
// Delete from End  
deleteEnd(head);  
cout << "\nAfter Deleting from End:\n";  
display(head);
```

```
return 0;  
}
```

Doubly

```

#include <iostream>
using namespace std;

// Node structure
struct Node
{
    int data;
    Node *prev;
    Node *next;
};

// Doubly Linked List Class
class DoublyLinkedList
{
private:
    Node *head;

public:
    DoublyLinkedList()
    {
        head = NULL;
    }

    // Insert at Beginning
    void insertBeginning(int value)
    {
        Node *newNode = new Node;
        newNode->data = value;
        newNode->prev = NULL;
        newNode->next = head;

        if (head != NULL)
            head->prev = newNode;

        head = newNode;
    }

    // Insert at End
    void insertEnd(int value)
    {
        Node *newNode = new Node;
        newNode->data = value;
        newNode->next = NULL;
    }

```

```

if (head == NULL)
{
    newNode->prev = NULL;
    head = newNode;
    return;
}

Node *temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
newNode->prev = temp;
}

// Insert After a Given Value (Middle)
void insertMiddle(int key, int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL)
    {
        cout << "Key not found.\n";
        return;
    }

    Node *newNode = new Node;
    newNode->data = value;

    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

```

```

    temp->next = newNode;
}

// Delete from Beginning
void deleteBeginning()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    delete temp;
}

// Delete from End
void deleteEnd()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    if (head->next == NULL)
    {
        delete head;
        head = NULL;
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;
    delete temp;
}

```

```

}

// Delete Middle (by Value)
void deleteMiddle(int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    while (temp != NULL && temp->data != value)
        temp = temp->next;

    if (temp == NULL)
    {
        cout << "Value not found.\n";
        return;
    }

    if (temp == head)
    {
        deleteBeginning();
        return;
    }

    if (temp->next == NULL)
    {
        deleteEnd();
        return;
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    delete temp;
}

// Forward Traversal
void traverseForward()
{
    if (head == NULL)

```

```

{
    cout << "List is empty.\n";
    return;
}

Node *temp = head;

cout << "Forward: ";

while (temp != NULL)
{
    cout << temp->data << " ";
    if (temp->next == NULL)
        break;
    temp = temp->next;
}

cout << endl;
}

// Backward Traversal
void traverseBackward()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    cout << "Backward: ";

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->prev;
    }

    cout << endl;
}

```

```

};

int main()
{
    DoublyLinkedList dll;
    int choice, value, key;

    do
    {
        cout << "\n===== Doubly Linked List =====\n";
        cout << "1. Insert at Beginning\n";
        cout << "2. Insert at Middle\n";
        cout << "3. Insert at End\n";
        cout << "4. Delete from Beginning\n";
        cout << "5. Delete from Middle\n";
        cout << "6. Delete from End\n";
        cout << "7. Traverse Forward\n";
        cout << "8. Traverse Backward\n";
        cout << "9. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "Enter value: ";
                cin >> value;
                dll.insertBeginning(value);
                break;

            case 2:
                cout << "Insert after value: ";
                cin >> key;
                cout << "Enter new value: ";
                cin >> value;
                dll.insertMiddle(key, value);
                break;

            case 3:
                cout << "Enter value: ";
                cin >> value;
                dll.insertEnd(value);
                break;

```

```
case 4:
    dll.deleteBeginning();
    break;

case 5:
    cout << "Enter value to delete: ";
    cin >> value;
    dll.deleteMiddle(value);
    break;

case 6:
    dll.deleteEnd();
    break;

case 7:
    dll.traverseForward();
    break;

case 8:
    dll.traverseBackward();
    break;

case 9:
    cout << "Program Ended.\n";
    break;

default:
    cout << "Invalid Choice!\n";
}

} while (choice != 9);

return 0;
}
```